LAPORAN PRAKTIKUM

STRUKTUR DATA

“Stack”

DISUSUN OLEH:

NOFRI ILHAM

2511531013

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

M Galid A

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data dengan materi *Stack* ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum sekaligus untuk memperdalam pemahaman penulis mengenai konsep struktur data, khususnya penggunaan *Stack* dalam pemrograman.

Dalam praktikum ini, penulis mempelajari konsep dasar *Stack* yang menggunakan prinsip LIFO (*Last In First Out*), serta bagaimana cara mengimplementasikan operasi-operasi dasar seperti *push*, *pop*, dan *peek* dalam program. Selain itu, praktikum ini juga memberikan pemahaman tentang penerapan *Stack* dalam berbagai kasus, seperti proses *undo/redo* dan pengelolaan data secara berurutan.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi penulis sendiri maupun pembaca.

Wassalamu'alaikum Warahmatullahi Wabarakatuh

Padang, 18 April 2026

Nofri Ilham

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Pengertian Stack 3
2.2 Karakteristik Stack 3
2.3 Impementasi Stack pada Praktikum 6
2.3.1 Kode Pertama(NilaiMaksimum_2511531013) 6
2.3.2 Kode Kedua(SiswaStack_2511531013) 7
2.3.3 Kode Ketiga(stackArray_2511531013) 10
2.3.4 Kode Keempat(StackPostfix_2511531013) 12
BAB III PENUTUP 14
3.1 Kesimpulan 14
3.2 Saran 14
DAFTAR PUSTAKA 16

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data yang terstruktur dan efisien sangat penting untuk mendukung kinerja suatu program. Struktur data merupakan konsep dasar dalam ilmu komputer yang digunakan untuk menyimpan dan mengatur data agar dapat diakses serta diproses dengan mudah. Salah satu jenis struktur data yang sering digunakan adalah *Stack*.

Stack adalah struktur data yang menggunakan prinsip LIFO (Last In First Out)**,** yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Struktur ini banyak digunakan dalam berbagai aplikasi, seperti manajemen memori, proses *undo-redo*, serta evaluasi ekspresi dalam pemrograman.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar *Stack*, cara kerjanya, serta implementasinya dalam pemrograman. Dengan memahami *Stack*, mahasiswa dapat meningkatkan kemampuan dalam memilih dan menggunakan struktur data yang sesuai dengan kebutuhan permasalahan.

1.2 Tujuan

Adapun tujuan dari praktikum struktur data dengan materi *Stack* adalah sebagai berikut:

1. Memahami konsep dasar struktur data *Stack*.
2. Mengetahui prinsip kerja *Stack* dengan metode LIFO (*Last In First Out*).
3. Mampu mengimplementasikan *Stack* dalam program.
4. Melatih kemampuan dalam melakukan operasi dasar *Stack* seperti *push*, *pop*, dan *peek*.

5. Meningkatkan pemahaman dalam penggunaan struktur data yang tepat dalam pemrograman.

1.3 Manfaat

Adapun manfaat dari praktikum struktur data dengan materi *Stack* adalah sebagai berikut:

1. Mahasiswa dapat memahami penggunaan struktur data *Stack* dalam pemrograman.
2. Meningkatkan kemampuan dalam mengelola data dengan metode LIFO.
3. Membantu mahasiswa dalam memahami penerapan *Stack* pada berbagai kasus seperti *undo/redo* dan evaluasi ekspresi.
4. Melatih keterampilan dalam penggunaan operasi dasar *Stack*.
5. Menambah wawasan mengenai pentingnya struktur data dalam pengembangan program.

BAB II
PEMBAHASAN

2.1 Pengertian Stack

Struktur Data Stack atau tumpukan adalah salah satu jenis struktur data pada komputer yang berfungsi untuk menyimpan dan mengorganisir data dalam urutan tertentu. Konsep utama dari stack adalah prinsip Last In First Out (LIFO), yang artinya data yang terakhir dimasukkan ke dalam stack akan menjadi data yang pertama kali diambil atau dikeluarkan dari stack.

Secara visual, stack dapat dibayangkan seperti tumpukan buku yang diletakkan satu per satu, di mana buku terakhir yang diletakkan akan menjadi buku yang paling mudah diambil atau diambil terlebih dahulu. Dalam implementasi pada program komputer, data pada stack disimpan pada satu tempat tertentu dalam memori komputer dan dapat diakses menggunakan operasi push (menambahkan data ke dalam stack) dan pop (mengeluarkan data dari stack).

Stack sering digunakan dalam implementasi algoritma dan fungsi matematis seperti evaluasi ekspresi matematis dan konversi infix ke postfix. Selain itu, stack juga digunakan dalam aplikasi yang membutuhkan manajemen tumpukan data seperti pengembangan sistem operasi, kompilator, dan perangkat lunak grafis.

2.2 Karakteristik Stack

Tentang jenis-jenis struktur data Stack, berikut penjelasan lebih detail untuk setiap jenisnya:

1. Stack yang Diimplementasikan Menggunakan Array

   Implementasi Stack menggunakan Array adalah metode yang paling sederhana, di mana data disimpan dalam sebuah array dengan akses ke data yang terakhir dimasukkan ke dalam stack. Data

dapat diambil atau dikeluarkan dari stack menggunakan operasi push dan pop pada indeks array yang sesuai.

Keuntungan dari implementasi Stack menggunakan Array adalah sederhana dan mudah dipahami, namun kelemahannya adalah kapasitas Stack dibatasi oleh ukuran array. Jika ukuran array terlalu kecil, maka Stack tidak dapat menampung banyak data dan jika ukuran array terlalu besar, maka akan memakan memori yang tidak efisien.

2. Stack yang Diimplementasikan Menggunakan Linked List

Implementasi Stack menggunakan Linked List memungkinkan untuk menambah atau mengurangi data dalam Stack tanpa terbatas oleh ukuran tertentu. Setiap data disimpan dalam sebuah simpul atau node dengan pointer ke simpul berikutnya. Data dapat diambil atau dikeluarkan dari Stack dengan menambah atau menghapus simpul pada ujung depan Linked List.

Keuntungan dari implementasi Stack menggunakan Linked List adalah tidak ada batasan kapasitas Stack dan dapat diatur sesuai kebutuhan. Namun, kelemahannya adalah kompleksitas implementasi yang lebih tinggi daripada implementasi Stack menggunakan Array.

3. Double-Ended Stack

Double-Ended Stack memungkinkan operasi push dan pop dilakukan pada kedua ujung Stack, yaitu ujung depan dan ujung belakang. Double-Ended Stack dapat diimplementasikan menggunakan Array atau Linked List.

Keuntungan dari Double-Ended Stack adalah memungkinkan akses data dari kedua ujung Stack, sehingga dapat digunakan untuk keperluan yang lebih fleksibel. Namun,

kelemahannya adalah kompleksitas implementasi yang lebih tinggi daripada implementasi Stack biasa.

4. Circular Stack

Circular Stack adalah Stack yang berbentuk sirkular, sehingga ketika Stack telah mencapai batas maksimum kapasitasnya, maka operasi push akan dilakukan pada posisi awal Stack. Circular Stack umumnya diimplementasikan menggunakan Array.

Keuntungan dari implementasi Circular Stack adalah tidak ada pemborosan memori dan penggunaan memori yang lebih efisien daripada implementasi Stack biasa. Namun, kelemahannya adalah kompleksitas implementasi yang lebih tinggi daripada implementasi Stack biasa.

5. Dynamic Stack

Dynamic Stack adalah Stack yang ukurannya dapat diubah secara dinamis selama program berjalan. Implementasi Dynamic Stack umumnya menggunakan Linked List.

Keuntungan dari implementasi Dynamic Stack adalah ukuran Stack dapat disesuaikan dengan kebutuhan, sehingga dapat menghemat penggunaan memori dan mengoptimalkan performa program. Namun, kelemahannya adalah kompleksitas implementasi yang lebih tinggi daripada implementasi Stack biasa.

6. Undo-Redo Stack

Undo-Redo Stack adalah Stack yang digunakan untuk melakukan operasi undo dan redo pada suatu aplikasi. Implementasi Undo-Redo Stack menggunakan Linked List, di mana setiap simpul pada Stack merepresentasikan perubahan yang dilakukan pada aplikasi.

Keuntungan dari implementasi Undo-Redo Stack adalah memungkinkan untuk mengembalikan perubahan yang sudah dilakukan pada aplikasi dengan mudah dan cepat menggunakan operasi undo dan redo. Namun, kelemahannya adalah kompleksitas implementasi yang lebih tinggi daripada implementasi Stack biasa dan memerlukan manajemen memori yang lebih kompleks.

2.3 Impementasi Stack pada Praktikum

Dalam praktikum Struktur Data pada pekan ketiga, mahasiswa diminta untuk mengulang penulisan kode yang telah disampaikan dan dicontohkan di papan tulis. Berikut ini adalah kodenya.

2.3.1 Kode Pertama(NilaiMaksimum_2511531013)

```java
package pekan3_2511531013;
import java.util.Stack;
public class NilaiMaksimum_2511531013 {
    public static int max (Stack <Integer>s_1013) {
    Stack <Integer> backup_1013 = new Stack <Integer>();
    int maxValue_1013 = s_1013.pop();
    backup_1013.push (maxValue_1013);
    while (!s_1013.isEmpty()) {
        int next_1013=s_1013.pop();
        backup_1013.push(next_1013);
        maxValue_1013 = Math.max(maxValue_1013, next_1013);
    }
    while (!backup_1013.isEmpty() ) {
        s_1013.push(backup_1013.pop());
    }
    return maxValue_1013;
  }
    public static void main(String[] args) {
        Stack <Integer> s_1013 = new Stack<Integer> ();
        s_1013.push (70);
        s_1013.push (12);
        s_1013.push (20);
        System.out.println("isi stack " + s_1013);
        System.out.println("Stack Teratas " + s_1013.peek());
        System.out.println("nilai maksimum " + max(s_1013));
    }

}
```

```
<terminated> NilaiMaksimum_
isi stack [70, 12, 20]
Stack Teratas 20
nilai maksimum 70
```

Kode Program 2.1

Program NilaiMaksimum_2511531013 ini digunakan untuk mencari nilai maksimum dari data yang disimpan di dalam struktur data *Stack*, tanpa mengubah isi asli dari *Stack* tersebut.

Pada method main, pertama dibuat sebuah *Stack* bernama s_1013 dengan tipe data Integer. Setelah itu, dimasukkan beberapa nilai menggunakan method push(), yaitu 70, 12, dan 20. Karena *Stack* menggunakan konsep LIFO (*Last In First Out*), maka elemen terakhir yang dimasukkan (yaitu 20) akan berada di posisi paling atas.

Kemudian program menampilkan isi *Stack* menggunakan System.out.println(), dan menampilkan elemen paling atas

menggunakan method peek(), yang hanya melihat elemen tanpa menghapusnya.

Selanjutnya program memanggil method max() untuk mencari nilai terbesar. Di dalam method ini, dibuat *Stack* tambahan bernama backup_1013 yang berfungsi sebagai tempat penyimpanan sementara. Hal ini diperlukan karena proses pencarian menggunakan pop() yang akan mengeluarkan elemen dari *Stack* utama.

Pertama, elemen paling atas diambil dan dijadikan sebagai nilai awal maksimum. Setelah itu dilakukan perulangan selama *Stack* masih berisi data. Setiap elemen di-*pop*, dimasukkan ke *backup*, lalu dibandingkan dengan nilai maksimum menggunakan Math.max() untuk mencari nilai terbesar.

2.3.2 Kode Kedua(SiswaStack_2511531013)

```java
package pekan3_2511531013;
import java.util.ArrayList;
class Siswa_2511531013 {
    String nama_1013;
    int nim_1013;

    public Siswa_2511531013 (String nama_1013, int nim_1013) {
        this.nama_1013= nama_1013;
        this.nim_1013= nim_1013;

    }

    @Override
    public String toString () {
        return "NIM: " + nim_1013 + ", Nama : " +nama_1013;
    }
}
public class SiswaStack_2511531013 {
    private ArrayList<Siswa_2511531013> stack_1013;

    public SiswaStack_2511531013() {
        stack_1013 = new ArrayList<>();
    }
    public void push (Siswa_2511531013 mhs) {
        stack_1013.add(mhs);
    }

public Siswa_2511531013 pop() {
    if (!isEmpty()) {
        return stack_1013.remove(stack_1013.size() -1);
     }
     return null;
    }
 public Siswa_2511531013 peek() {
     if (!isEmpty()) {
         return stack_1013.get(stack_1013.size() -1 );
     }
     return null;
 }

 public boolean isEmpty () {
```

```java
    return stack_1013.isEmpty();
    }
public void tampilkanSiswa_1013() {
    for (int i= stack_1013.size() -1; i>=0; i--) {
        System.out.println(stack_1013.get(i));
    }
}
public static void main (String [] args) {
    SiswaStack_2511531013 studentstack = new SiswaStack_2511531013();

    Siswa_2511531013 mhs1 = new Siswa_2511531013 ("Ali ", 1);
    Siswa_2511531013 mhs2 = new Siswa_2511531013 ("Boby ", 2);
    Siswa_2511531013 mhs3 = new Siswa_2511531013 ("Charles ", 3);

    studentstack.push(mhs1);
    studentstack.push(mhs2);
    studentstack.push(mhs3);

    System.out.println("siswa di dalam stack : ");
    studentstack.tampilkanSiswa_1013();

    System.out.println("siswa teratas " + studentstack.peek());
    System.out.println("mengeluarkan siswa teratas dari stack " + studentstack.pop());
    System.out.println("daftar siswa setelah di pop : ");
    studentstack.tampilkanSiswa_1013();
}

}
```

Kode Program 2.2

Program SiswaStack_2511531013 diatas bertujuan untuk mengelola data siswa dengan konsep LIFO (*Last In First Out*), yaitu data yang terakhir masuk akan menjadi yang pertama keluar.

Pada bagian awal, terdapat kelas Siswa_2511531013 yang digunakan sebagai model data. Kelas ini memiliki dua atribut, yaitu nama_1013 dan nim_1013. Konstruktor digunakan untuk mengisi nilai atribut saat objek dibuat, dan method toString() dioverride agar data siswa dapat ditampilkan dengan format yang lebih rapi.

Selanjutnya, pada kelas SiswaStack_2511531013, dibuat sebuah ArrayList bernama stack_1013 yang berfungsi sebagai wadah penyimpanan data siswa. Meskipun menggunakan ArrayList, cara penggunaannya diatur seperti *Stack*.

Beberapa method yang terdapat dalam kelas ini antara lain:

- push()
  Digunakan untuk menambahkan data siswa ke dalam *Stack* (bagian paling atas).

- pop()
  Digunakan untuk menghapus dan mengembalikan data

siswa yang berada di posisi teratas. Jika *Stack* kosong, maka akan mengembalikan null.

- peek()
  Digunakan untuk melihat data siswa yang berada di posisi teratas tanpa menghapusnya.
- isEmpty()
  Digunakan untuk mengecek apakah *Stack* dalam keadaan kosong atau tidak.
- tampilkanSiswa_1013()
  Digunakan untuk menampilkan seluruh isi *Stack* dari atas ke bawah dengan menggunakan perulangan.

Pada method main, dibuat tiga objek siswa yaitu Ali, Boby, dan Charles. Ketiga data tersebut dimasukkan ke dalam *Stack* menggunakan method push(). Setelah itu, program menampilkan seluruh isi *Stack*, menampilkan siswa teratas menggunakan peek(), kemudian menghapus siswa teratas menggunakan pop(), dan terakhir menampilkan kembali isi *Stack* setelah penghapusan.

2.3.3 Kode Ketiga(stackArray_2511531013)

```java
package pekan3_2511531013;

public class stackArray_2511531013 {
    static final int MAX_1013= 1000;
    int top_1013;
    int a[] = new int [MAX_1013];
    boolean isEmpty_1013()
    {
        return (top_1013 < 0) ;
    }
    stackArray_2511531013()
    {
        top_1013=-1;
    }
    boolean push_1013 (int x)
    {
        if (top_1013>= (MAX_1013 - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a[++top_1013] = x;
            System.out.println( x  + " dimasukkan dalam stack");
            return true;
        }
    }
    int pop_1013()
    {
        if (top_1013 < 0 ) {
            System.out.println("stack underflow");
            return 0;
        }else {
            int x = a[top_1013--];
            return x;
        }
    }
    int peek_1013()
    {
        if (top_1013<0)
        {
            System.out.println("stack underflow");
```

```java
            return 0;
        }
        else
        {
            int x = a [top_1013];
            return x;
        }
    }
    void print_1013 () {
        for (int i=top_1013; i>-1;i--) {
            System.out.print(" " +a[i]);
        }
    }

}
```

Kode program 2.3

Program stackArray_2511531013 merupakan implementasi struktur data *Stack* menggunakan array dalam bahasa Java. Pada program ini, digunakan variabel MAX_1013 sebagai kapasitas

maksimum *Stack*, yaitu sebanyak 1000 elemen. Data disimpan dalam array a[], sedangkan variabel top_1013 digunakan untuk menunjukkan posisi elemen paling atas (*top*). Nilai awal top_1013 adalah -1 yang menandakan bahwa *Stack* masih kosong.

Method isEmpty_1013() digunakan untuk mengecek apakah *Stack* kosong, yaitu ketika nilai top_1013 kurang dari 0. Method push_1013(int x) digunakan untuk menambahkan elemen ke dalam *Stack*. Sebelum menambahkan, program akan mengecek apakah *Stack* sudah penuh (*overflow*). Jika belum penuh, maka nilai top_1013 akan dinaikkan terlebih dahulu, kemudian data dimasukkan ke dalam array.

Method pop_1013() digunakan untuk mengambil dan menghapus elemen paling atas dari *Stack*. Jika *Stack* kosong (*underflow*), maka akan ditampilkan pesan kesalahan. Jika tidak kosong, maka elemen teratas diambil dan nilai top_1013 dikurangi.

Method peek_1013() digunakan untuk melihat elemen paling atas tanpa menghapusnya. Jika *Stack* kosong, maka akan muncul pesan *underflow*. Sedangkan method print_1013() digunakan untuk menampilkan seluruh isi *Stack* dari atas ke bawah menggunakan perulangan.

```
1  package pekan3_2511531013;
2
3  public class stackArrayDriver_2511531013 {
4
5      public static void main(String[] args) {
6          stackArray_2511531013 s = new stackArray_2511531013 ();
7          s.push_1013(10);
8          s.push_1013(20);
9          s.push_1013(30);
10         System.out.println(s.pop_1013() + " dikeluarkan dari stack");
11         System.out.println("elemen teratas adalah : " + s.peek_1013());
12         System.out.print("element pada stack    :");
13         s.print_1013();
14
15     }
16
17 }
```

```
<terminated> stackArrayDriver_2511531013
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
elemen teratas adalah : 20
element pada stack    : 20 10
```

Kode Program 2.4

Program stackArrayDriver_2511531013 merupakan kelas *driver* yang digunakan untuk menguji kelas sebelumnya

Pada method main, pertama dibuat sebuah objek *Stack* bernama s. Selanjutnya dilakukan beberapa operasi dasar *Stack*, yaitu menambahkan elemen ke dalam *Stack* menggunakan method push_1013() dengan nilai 10, 20, dan 30. Karena menggunakan konsep LIFO (*Last In First Out*), maka elemen terakhir yang dimasukkan (30) berada di posisi paling atas.

Kemudian program memanggil method pop_1013() untuk menghapus dan mengambil elemen teratas dari *Stack*. Elemen yang dikeluarkan adalah 30, dan nilai tersebut langsung ditampilkan ke layar.

Setelah itu, program menggunakan method peek_1013() untuk melihat elemen teratas yang tersisa tanpa menghapusnya. Karena sebelumnya 30 sudah di-*pop*, maka elemen teratas sekarang adalah 20.

Terakhir, method print_1013() digunakan untuk menampilkan seluruh isi *Stack*. Data akan ditampilkan dari atas ke bawah, yaitu dimulai dari elemen teratas hingga elemen terbawah.

2.3.4 Kode Keempat(StackPostfix_2511531013)

```
1  package pekan3_2511531013;
2  import java.util.Scanner;
3  import java.util.Stack;
4
5  public class StackPostfix_2511531013 {
6
7      public static int postfixEvaluate_1013 (String expression) {
8          Stack <Integer> s = new Stack <Integer>();
9          Scanner input_1013 = new Scanner (expression);
10         while (input_1013.hasNext()) {
11             if (input_1013.hasNextInt()) {  //an operan (integer)
12                 s.push(input_1013.nextInt());
13             } else {                        //an operator
14                 String operator_1013 = input_1013.next();
15                 int operand2 = s.pop();
16                 int operand1 = s.pop();
17                 if (operator_1013.equals("+")) {
18                     s.push (operand1+operand2);
19                 } else if (operator_1013.equals("-")) {
20                     s.push(operand1-operand2);
21                 } else if (operator_1013.equals("*")) {
22                     s.push(operand1*operand2);
23                 } else {
24                     s.push(operand1/operand2);
25                 }
26             }
27         }
28         input_1013.close();
29         return s.pop();
30     }
31     public static void main (String [] args) {
32         System.out.println("hasil postfix = " + postfixEvaluate_1013("5 2 5 * + 7 -"));
33     }
34
35 }
```

```
<terminated> StackPostfix_2
hasil postfix = 8
```

Program diatas digunakan untuk menghitung hasil dari ekspresi aritmatika dalam bentuk *postfix* (Reverse Polish Notation) menggunakan struktur data *Stack*.

Pada method postfixEvaluate_1013(String expression), dibuat sebuah *Stack* bertipe Integer yang digunakan untuk menyimpan operand (angka). Selain itu, digunakan objek Scanner untuk membaca setiap token (angka atau operator) dari string ekspresi yang diberikan.

Proses dimulai dengan perulangan while selama masih ada token dalam ekspresi. Jika token yang dibaca berupa angka (hasNextInt()), maka angka tersebut langsung dimasukkan ke dalam *Stack* menggunakan push().

Jika token berupa operator, maka program akan mengambil dua nilai teratas dari *Stack* menggunakan pop(). Nilai pertama yang diambil menjadi operand2, dan nilai kedua menjadi operand1. Setelah itu, dilakukan operasi sesuai dengan operator yang dibaca, seperti penjumlahan (+), pengurangan (-), perkalian (*), atau pembagian (/). Hasil dari operasi tersebut kemudian dimasukkan kembali ke dalam *Stack*.

Proses ini terus berlanjut hingga semua token dalam ekspresi selesai diproses. Setelah itu, hasil akhir yang tersisa di dalam *Stack* diambil menggunakan pop() dan dikembalikan sebagai hasil perhitungan.

Pada method main, program memanggil fungsi postfixEvaluate_1013() dengan contoh ekspresi "5 2 5 * + 7 -".

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Struktur Data dengan materi *Stack* yang telah dilakukan, dapat disimpulkan bahwa *Stack* merupakan struktur data yang bekerja dengan prinsip LIFO (*Last In First Out*), di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Dalam praktikum ini, penulis memahami berbagai operasi dasar pada *Stack* seperti *push*, *pop*, dan *peek*, serta bagaimana cara mengimplementasikannya dalam program.

Selain itu, praktikum ini juga menunjukkan bahwa *Stack* dapat diimplementasikan dengan berbagai cara, seperti menggunakan array, *ArrayList*, maupun menggunakan library bawaan Java. Setiap metode memiliki kelebihan dan kekurangan masing-masing, namun semuanya tetap mengikuti konsep dasar *Stack*.

Melalui beberapa contoh program yang telah dibuat, seperti pencarian nilai maksimum, pengelolaan data siswa, serta evaluasi ekspresi postfix, dapat dilihat bahwa *Stack* memiliki banyak penerapan dalam pemrograman. Secara keseluruhan, praktikum ini membantu meningkatkan pemahaman tentang struktur data *Stack* serta pentingnya penggunaannya dalam menyelesaikan berbagai permasalahan dalam pemrograman.

3.2 Saran

Adapun saran yang dapat diberikan setelah pelaksanaan praktikum ini adalah sebagai berikut:

1. Mahasiswa diharapkan lebih sering berlatih menggunakan *Stack* dalam berbagai kasus agar pemahaman semakin mendalam.

2. Praktikum selanjutnya sebaiknya diberikan contoh kasus yang lebih kompleks agar mahasiswa dapat lebih memahami penerapan *Stack* dalam dunia nyata.
3. Mahasiswa disarankan untuk mempelajari struktur data lain seperti *Queue*, *LinkedList*, dan *Tree* sebagai perbandingan dengan *Stack*.
4. Dalam pembuatan program, sebaiknya ditambahkan validasi agar dapat menghindari kesalahan seperti *stack overflow* dan *underflow*.

DAFTAR PUSTAKA

[1] Rini Rahmawati, “Struktur Data Stack: Pengertian, Jenis dan Contoh,” [Daring]. Tersedia pada: https://dosenit.com/ilmu-komputer/struktur-data-stack [Diakses: 18-Apr-2026].

LAPORAN TUGAS PEKAN 2

STRUKTUR DATA

DISUSUN OLEH:

NOFRI ILHAM

2511531013

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

Topik : Implementasi Stack (LIFO) menggunakan java.util.Stack

Tema : Simulasi Browser History

1. Deskripsi Tugas

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

2. Aturan Penamaan (Wajib Dipatuhi)

1) Nama Kelas : Gunakan akhiran NIM lengkap.
   Contoh : Website_2311532008 dan Browser_2311532008.
2) Nama Variabel Atribut : Setiap variabel atribut di kelas ADT gunakan akhiran 4 digit terakhir NIM.
   Contoh (NIM ...2008): String judul_2008, String url_2008.

3. Struktur Program

1) Kelas ADT Website (Website_NIMLengkap)

Atribut :

- Judul Website

- URL Website

Method :

- Constructor untuk menginisialisasi atribut. 2) Kelas Driver (Browser_NIMLengkap)

- Kunjungi Website (Push) : Memasukkan data website baru yang sedang

dikunjungi ke dalam tumpukan.

- Tombol Back (Pop) : Menghapus halaman website terakhir (paling atas)

dan menampilkan judul halaman yang berhasil dihapus.

- Lihat Halaman Saat Ini (Peek) : Menampilkan detail website yang sedang

aktif dibuka tanpa menghapusnya dari tumpukan.

- Cek Status History : Menampilkan jumlah total riwayat yang tersimpan

atau mengecek apakah riwayat kosong menggunakan .isEmpty().

4. Contoh Tampilan Output Program

=== Browser History NIM: 2411532007 ===

1. Kunjungi Website (Push)

2. Tombol Back (Pop)

3. Lihat Halaman Aktif (Peek)

4. Keluar

Pilihan: 1

Masukkan Judul: SSO UNAND

Masukkan URL: https://sso.unand.ac.id/login

Berhasil mengunjungi halaman!

Jawaban:

1.Class ADT (Website_2511531013)

```java
package Pekan3_2511531013;

public class Website_2511531013 {
    private String judul_1013;
    private String url_1013;

    // Konstructor
    Website_2511531013(String judul_1013, String url_1013) {
        this.judul_1013 = judul_1013;
        this.url_1013 = url_1013;
    }

    // Getter
    public String getJudul_1013() {
        return judul_1013;
    }

    public String getUrl_1013() {
        return url_1013;
    }

    // Setter
    public void setJudul_1013(String judul_1013) {
        this.judul_1013 = judul_1013;
    }

    public void setUrl_1013(String url_1013) {
        this.url_1013 = url_1013;
    }

    // Override
    @Override
    public String toString() {
        return "Judul : " + judul_1013 + "\nURL   : " + url_1013;
    }
}
```

Constructor

Website_2511531013(String judul_1013, String url_1013)

Constructor digunakan untuk menginisialisasi nilai atribut judul_1013 dan url_1013 saat objek Website dibuat. Nilai parameter akan disimpan ke atribut menggunakan keyword this.

Getter

Public String getJudul_1013()

Mengembalikan nilai atribut judul_1013 (judul website) agar dapat diakses dari luar class.

Public String getUrl_1013()

Mengembalikan nilai atribut url_1013 (alamat website).

Setter

Public void setJudul_1013(String judul_1013)

Digunakan untuk mengubah nilai atribut judul_1013 dengan nilai baru yang diberikan.

Public void setUrl_1013(String url_1013)

Digunakan untuk mengubah nilai atribut url_1013.

Method toString()

@Override

Public String toString()

Method ini di-override dari class Object dan berfungsi untuk mengembalikan representasi string dari objek Website, yaitu berisi judul dan URL dalam format yang rapi. Method ini otomatis dipanggil saat objek dicetak menggunakan System.out.println().

2.Kelas Driver(Browser_2511531013)

Musik_2511531013.java | Playlist_2511531013.java | Website_2511531013.java | *Browser_2511531013.ja

```java
package Pekan3_2511531013;
import java.util.Scanner;
import java.util.Stack;
public class Browser_2511531013 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Stack<Website_2511531013> history_1013 = new Stack<>();
        int pilihan_1013;
        do {
            System.out.println("\n=== Browser History NIM: 2511531013 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Keluar");
            System.out.print("Pilihan: ");
            pilihan_1013 = input.nextInt();
            input.nextLine();
            switch (pilihan_1013) {
                case 1:
                    System.out.print("Masukkan Judul: ");
                    String judul_1013 = input.nextLine();

                    System.out.print("Masukkan URL: ");
                    String url_1013 = input.nextLine();

                    Website_2511531013 web = new Website_2511531013(judul_1013, url_1013);
                    history_1013.push(web);

                    System.out.println("Berhasil mengunjungi halaman!");
                    break;

                case 2:
                    if (!history_1013.isEmpty()) {
                        Website_2511531013 hapus = history_1013.pop();
                        System.out.println("Kembali dari: " + hapus.getJudul_1013());
                    } else {
                        System.out.println("History kosong!");
                    }
                    break;

                case 3:
                    if (!history_1013.isEmpty()) {
                        Website_2511531013 atas = history_1013.peek();
                        System.out.println("Halaman Aktif:");
                        System.out.println("Judul: " + atas.getJudul_1013());
                        System.out.println("URL: " + atas.getUrl_1013());
                    } else {
                        System.out.println("Tidak ada halaman aktif!");
                    }
                    break;

                case 4:
                    System.out.println("Keluar dari website");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan_1013 != 4);

        input.close();
    }
}
```

Import

Import java.util.Scanner;

Import java.util.Stack;

Digunakan untuk mengimpor class Scanner sebagai alat input dari user dan Stack sebagai struktur data untuk menyimpan history browser dengan konsep LIFO (Last In First Out).

Method main()

Public static void main(String[] args)

Merupakan titik awal eksekusi program. Di dalam method ini dibuat objek Scanner untuk input dan Stack untuk menyimpan data website yang dikunjungi.

Inisialisasi Stack

Stack<Website_2511531013> history_1013 = new Stack<>();

Digunakan untuk membuat stack bernama history_1013 yang menyimpan objek Website. Stack ini berfungsi sebagai history browser.

Menu Program

System.out.println("1. Kunjungi Website (Push)");

System.out.println("2. Tombol Back (Pop)");

System.out.println("3. Lihat Halaman Aktif (Peek)");

System.out.println("4. Keluar");

Digunakan untuk menampilkan pilihan menu yang bisa dipilih oleh user.

Case 1 (Push)

History_1013.push(web);

Digunakan untuk menambahkan website ke dalam stack. Data website diinput oleh user, kemudian dibuat objek Website dan dimasukkan ke stack menggunakan method push().

Case 2 (Pop)

Website_2511531013 hapus = history_1013.pop();

Digunakan untuk menghapus halaman terakhir yang dikunjungi (elemen teratas stack). Method pop() akan menghapus sekaligus mengembalikan data tersebut.

Case 3 (Peek)

Website_2511531013 atas = history_1013.peek();

Digunakan untuk melihat halaman yang sedang aktif tanpa menghapusnya dari stack. Method peek() hanya mengambil elemen teratas.

Perulangan do-while

} while (pilihan_1013 != 4);

Digunakan agar menu terus tampil berulang sampai user memilih opsi keluar (4).

Penutup

Input.close();

Digunakan untuk menutup objek Scanner agar tidak terjadi kebocoran resource.

3.Output

```
<terminated> Browser_2511531013 [Java Application] C:\Use

=== Browser History NIM: 2511531013 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531013 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3
Halaman Aktif:
Judul: SSO UNAND
URL: https://sso.unand.ac.id/login

=== Browser History NIM: 2511531013 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 2
Kembali dari: SSO UNAND

=== Browser History NIM: 2511531013 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 4
Keluar dari website
```